

Backend Developer Case Study

Valizu Backend API

Teknik Değerlendirme Dokümanı

Aday: Okhtay Sattari

Pozisyon: Backend Developer

Proje: Valizu Backend Case Study

API Versiyonu: 1.0.0

OpenAPI Versiyonu: 3.0.3

1. Yönetici Özeti

Bu proje, Valizu backend case study kapsamında geliştirilmiş bir REST API uygulamasıdır.

Temel amaç aşağıdaki domain kaynaklarının yönetilebildiği güvenilir bir backend API geliştirmektir:

- Users
- Trips
- Bags
- Items
- Bag içerisindeki Items

CRUD operasyonlarının yanında case study'nin en önemli teknik gereksinimlerinden biri olan **Trip Copy operasyonunun concurrent request'ler altında güvenli ve idempotent şekilde gerçekleştirilmesine** özellikle önem verilmiştir.

Uygulamada:

- PostgreSQL
- Prisma
- TypeScript
- Express.js
- Zod
- OpenAPI
- Swagger UI

- Vitest

kullanılmıştır.

Copy Trip operasyonunda:

- database-level unique constraint
- SHA-256 request hash
- transaction
- idempotency response storage
- Prisma unique constraint error handling
- concurrency testleri

birlikte kullanılmaktadır.

Temel tasarım prensibi şudur:

Concurrency altında doğruluk yalnızca application-level kontrol ile değil, database-level constraint'ler ile garanti edilmelidir.

2. Teknoloji Stack'i

Teknoloji	Kullanım Alanı
Node.js	Runtime
TypeScript	Uygulama dili
Express.js	HTTP API framework
PostgreSQL	Relational database
Prisma	ORM ve database erişimi
Zod	Request validation
OpenAPI 3.0.3	API contract
Swagger UI	API dokümantasyonu
Vitest	Automated testing

pnpm Package management

Docker / Docker Compose Containerization

3. Mimari

Uygulama modüler bir backend mimarisi kullanmaktadır.

Temel katmanlar:

1. HTTP/API layer
2. Validation layer
3. Business/service layer
4. Persistence layer
5. Database

Genel akış:

HTTP Request



Express Route



Validation / Authentication



Controller



Service



Prisma



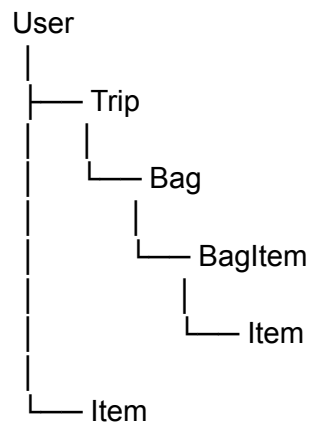
PostgreSQL

Business logic service katmanında tutulmaktadır.

Bu özellikle Copy Trip operasyonu için önemlidir. Idempotency, transaction boundary, ownership kontrolü ve resource copying gibi kuralların HTTP controller içerisine dağıtılması yerine service katmanında merkezi şekilde uygulanması daha doğru bir tasarım sağlar.

4. Domain Model

Domain aşağıdaki ilişkiler üzerinden modellenmiştir:



Bir User:

- Trips
- Items

sahibidir.

Bir Trip:

- Bags

içerir.

Bir Bag:

- BagItems

içerir.

BagItem ise:

- bir Bag
- bir Item

arasındaki ilişkiyi temsil eder ve ayrıca:

- quantity

bilgisini taşır.

5. Database Tasarımı

5.1 User

User entity API kullanıcılarını temsil eder.

Temel constraint:

id UUID PRIMARY KEY
email UNIQUE

Email database seviyesinde unique olarak tanımlanmıştır.

5.2 Trip

Trip bir User'a aittir.

Temel alanlar:

id
userId
name
destination
startDate
endDate
createdAt
updatedAt

`userId` üzerinde index bulunmaktadır.

Bu sayede kullanıcının trip'lerinin verimli şekilde listelenmesi sağlanır.

5.3 Bag

Bag bir Trip'e aittir.

Temel alanlar:

id
tripId
name
createdAt
updatedAt

`tripId` üzerinde index bulunmaktadır.

5.4 Item

Item bir User'a aittir.

Temel alanlar:

id
userId
name
category
createdAt
updatedAt

`userId` üzerinde index bulunmaktadır.

5.5 BagItem

BagItem, Bag ve Item arasındaki ilişkiyi temsil eder.

Model:

bagId
itemId
quantity

Database aşağıdaki constraint'i uygular:

UNIQUE(bagId, itemId)

Böylece aynı item aynı bag içerisinde birden fazla ayrı BagItem kaydı olarak oluşturulamaz.

Quantity ilişkiye ait olduğu için BagItem üzerinde tutulmaktadır.

6. Ownership ve Authorization

Case study kapsamında authentication mekanizması basitleştirilmiş olarak:

X-User-Id

header'ı üzerinden uygulanmıştır.

Örneğin:

X-User-Id: 11111111-1111-4111-8111-111111111111

Buradaki önemli tasarım kararı ownership kontrolünün yalnızca controller seviyesinde yapılmamasıdır.

Database sorguları uygun yerlerde authenticated user ile sınırlandırılmaktadır.

Örneğin Copy Trip sırasında source trip:

id = sourceTripId
AND
userId = authenticatedUserId

koşullarıyla alınmaktadır.

Dolayısıyla başka bir kullanıcının Trip UUID'sini bilmek o kaynağa erişmek için yeterli değildir.

Bu yaklaşım object-level authorization problemlerinin ve IDOR benzeri erişim açıklarının önlenmesine yardımcı olur.

7. API Endpoint'leri

Health

GET /health

API health durumunu döndürür.

Users

GET /users

Case study dataset içerisindeki kullanıcıları listeler.

Trips

GET /trips

Authenticated user'a ait trip'leri listeler.

POST /trips/{tripId}/copy

Trip'i bags ve bag items ile birlikte kopyalar.

Bags

GET /trips/{tripId}/bags
POST /trips/{tripId}/bags
PATCH /bags/{bagId}

Bag listeleme, oluşturma ve güncelleme operasyonlarını sağlar.

Bag Items

GET /bags/{bagId}/items
POST /bags/{bagId}/items
DELETE /bags/{bagId}/items/{itemId}

Add Item operasyonunda item zaten bag içerisinde bulunuyorsa quantity artırılır.

Items

GET /items

Authenticated user'a ait item'ları listeler.

8. OpenAPI

API OpenAPI 3.0.3 ile dokümente edilmiştir.

OpenAPI specification aşağıdakileri tanımlar:

- endpoints
- path parameters
- request headers
- request bodies
- authentication
- validation requirements
- response schemas
- error schemas

- HTTP status codes

OpenAPI source:

src/openapi.ts

Idempotency-Key header'ı da API contract içerisinde açıkça tanımlanmıştır.

9. Copy Trip Gereksinimi

Copy Trip operasyonu uygulamanın en önemli bölümüdür.

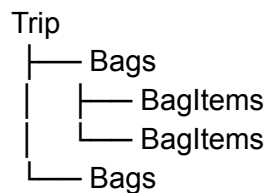
Endpoint:

POST /trips/{tripId}/copy

Zorunlu header:

Idempotency-Key: <key>

Operasyon aşağıdaki yapıyı kopyalar:



Yeni Trip yeni bir UUID alır.

Yeni Trip adı:

Copy of <source trip name>

Diğer Trip bilgileri source Trip'ten kopyalanır.

10. Idempotency Tasarımı

Idempotency, aynı logical request'in tekrar gönderilmesi halinde birden fazla side effect oluşmaması anlamına gelir.

Örneğin:

```
POST /trips/123/copy
Idempotency-Key: abc-123
```

İlk çağrıda bir Trip oluşturur.

Aynı request tekrar:

```
POST /trips/123/copy
Idempotency-Key: abc-123
```

şeklinde gönderilirse yeni bir Trip oluşturulmaz.

İlk operasyonun sonucu tekrar döndürülür.

11. Neden Sadece Application-Level Kontrol Yeterli Değildir?

Basit bir implementation şöyle olabilir:

```
SELECT idempotency record
```

if record does not exist:

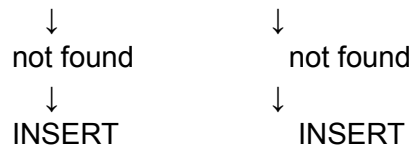
```
  INSERT idempotency record
  copy trip
```

Bu yaklaşım race condition'a açıktır.

Örneğin:

Request A	Request B
-----------	-----------

SELECT	SELECT
--------	--------



Her iki request de diğer request insert işlemini gerçekleştirmeden önce kaydın olmadığını görebilir.

Sonuç olarak iki ayrı Trip copy oluşturulabilir.

Bu nedenle:

check → then create

tek başına yeterli değildir.

12. Database-Level Concurrency Guarantee

Prisma schema içerisinde:

```
@@unique([userId, operation, key])
```

tanımlanmıştır.

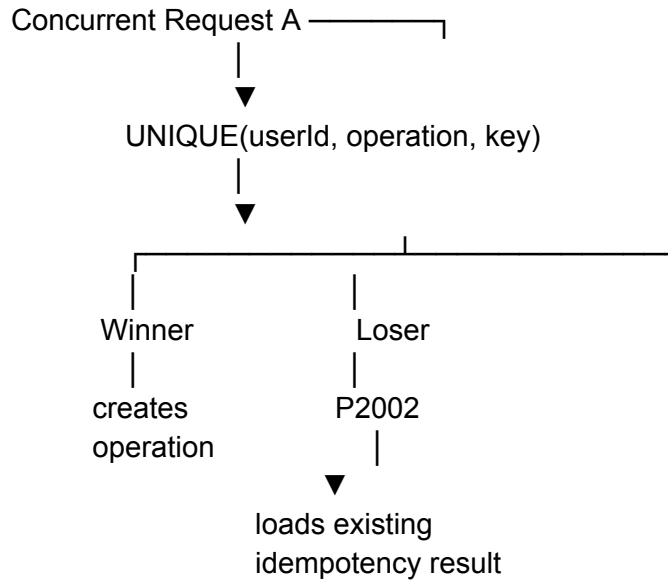
Bu database seviyesinde:

```
UNIQUE(userId, operation, key)
```

constraint'i oluşturur.

Böylece database concurrency konusunda authoritative boundary haline gelir.

Genel akış:



Bu yaklaşım birden fazla application instance çalışırken dahi geçerliliğini korur.

13. Transaction Tasarımı

Idempotency kaydı ve kopyalanan resource'lar aynı database transaction içerisinde oluşturulmaktadır.

Transaction sırası:

1. IdempotencyKey oluştur
2. Source Trip'i getir
3. Copied Trip oluştur
4. Copied Bags oluştur
5. Copied BagItems oluştur
6. ResponseBody kaydet
7. ResourceId kaydet
8. Commit

Herhangi bir işlem başarısız olursa:

ROLLBACK

gerçekleşir.

Transaction içerisindeki bütün değişiklikler geri alınır.

14. Failure Semantics

Örneğin olmayan bir Trip için:

POST /trips/{non-existent-trip}/copy

Idempotency-Key: abc-123

gönderildiğinde:

404 TRIP_NOT_FOUND

döner.

Transaction rollback olduğu için IdempotencyKey kaydı da geri alınır.

Sonuç:

Idempotency-Key abc-123

başarısız request tarafından tüketilmiş olmaz.

Client aynı key ile daha sonra retry gerçekleştirebilir.

Bu bilinçli bir tasarım kararıdır.

15. Concurrent Request Yönetimi

Service aşağıdaki Prisma hatasını özel olarak yakalar:

Prisma.PrismaClientKnownRequestError

ve:

P2002

kodunu kontrol eder.

P2002 unique constraint ihlalini ifade eder.

Uygulama bütün database hatalarını idempotency conflict olarak değerlendirmez.

Yalnızca beklenen unique constraint ihlali idempotency recovery mekanizmasına yönlendirilir.

Bu yaklaşım unrelated database hatalarının yanlışlıkla başarılı idempotency response'una dönüştürülmesini engeller.

16. Request Hash

Service aşağıdaki logical request bilgisinden SHA-256 hash üretir:

COPY_TRIP:<sourceTripId>

Bu hash IdempotencyKey kaydında saklanır.

Böylece aynı idempotency key'in başka bir logical request için kullanılması tespit edilebilir.

Örneğin:

İlk request:

Idempotency-Key: abc-123

Source Trip: A

Daha sonra:

Idempotency-Key: abc-123

Source Trip: B

gönderilirse request hash'leri farklı olur.

API:

409 IDEMPOTENCY_KEY_REUSED

döndürür.

Bu sayede aynı key farklı operasyonları temsil etmek için kullanılamaz.

17. Response'un Saklanması

Copy Trip başarıyla tamamlandığında sonuç:

`IdempotencyKey.responseBody`

alanında saklanır.

Oluşturulan resource ID:

`IdempotencyKey.resourceId`

alanında saklanır.

Böylece retry sırasında aynı sonuç doğrudan döndürülebilir.

Bunun iki önemli avantajı vardır:

1. Duplicate resource oluşturulmaz.
 2. Retry, ilk request ile aynı logical sonucu döndürür.
-

18. try/catch Neden Kullanılıyor?

`try/catch` önemlidir ancak concurrency mekanizmasının kendisi değildir.

Asıl concurrency guarantee:

Database UNIQUE constraint

tarafından sağlanır.

`try/catch` ise bu constraint'in beklenen sonucunu application-level davranışa dönüştürür.

Akış:

Database constraint

↓
Duplicate idempotency records engellenir
↓
Prisma losing transaction'a P2002 döndürür
↓
Service P2002'yi yakalar
↓
Mevcut completed result bulunur
↓
Original result döndürülür

Dolayısıyla concurrency control ile error handling birbirinden farklı sorumluluklardır.

19. Automated Testing

Vitest ile Copy Trip için kapsamlı testler hazırlanmıştır.

Test dosyası:

tests/trips/copy.trip.test.ts

Testler dört önemli senaryoyu kapsar.

19.1 Concurrent Requests

İki request aynı anda çalıştırılır:

```
Promise.all([
  copyTrip(...),
  copyTrip(...),
])
```

Test aşağıdakileri doğrular:

- iki çağrı aynı sonucu döndürür
- iki ID aynıdır
- yalnızca bir copied Trip oluşur
- yalnızca bir idempotency record oluşur

- idempotency record COMPLETED durumundadır
- resourceId copied Trip'i gösterir

Bu test doğrudan concurrency gereksinimini doğrular.

19.2 Idempotency Key Reuse

Önce Trip A kopyalanır.

Daha sonra aynı key Trip B için kullanılmaya çalışılır.

Beklenen sonuç:

409

IDEMPOTENCY_KEY_REUSED

Bu test request hash mekanizmasını doğrular.

19.3 Başarısız Copy Key'i Tüketmez

Olmayan bir Trip kopyalanmaya çalışılır.

Beklenen sonuç:

404

TRIP_NOT_FOUND

Ardından IdempotencyKey kaydının bulunmadığı kontrol edilir.

Bu test transaction rollback davranışını doğrular.

19.4 Completed Request Retry

Aynı copy request aynı key ile iki kez gönderilir.

Test:

second === first

koşulunu doğrular.

Ayrıca yalnızca bir copied Trip olduğu kontrol edilir.

20. Seed Stratejisi

Projede deterministic seed mekanizması bulunmaktadır.

Seed dosyası:

src/scripts/seed.ts

sabit UUID'ler ve Prisma upsert operasyonları kullanır.

Bu nedenle seed:

- repeatable
- deterministic
- birden fazla kez çalıştırılabilir

durumdadır.

Dataset:

User

└─ Valizu Case User

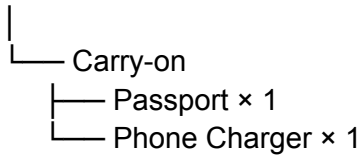
Items

└─ Passport

└─ Phone Charger

Trip

└─ Istanbul Trip



Seed user:

case@valizu.test

21. Validation

Request validation için Zod kullanılmaktadır.

Örnek validation kuralları:

- bag name length
- required fields
- UUID format
- quantity minimum
- Idempotency-Key length

Validation error'ları business error'larından ayrılmıştır.

Örneğin:

400 VALIDATION_ERROR

ile:

404 TRIP_NOT_FOUND

birbirinden ayrıdır.

Bu durum API client'larının daha öngörülebilir şekilde hata yönetmesini sağlar.

22. Error Handling

API structured error response kullanır.

Örneğin:

```
{  
  "error": "TRIP_NOT_FOUND",  
  "message": "Trip not found"  
}
```

Validation error'ları ayrıca:

details

alanını içerir.

Bu error modelleri OpenAPI içerisinde de dokümente edilmiştir.

23. Referential Integrity

Database foreign key ve cascade kuralları kullanmaktadır.

Örneğin:

```
User  
├── Trip  
│   ├── Bag  
│   │   └── BagItem
```

uygun parent silme işlemlerinde dependent resource'ların cascade edilmesi sağlanır.

BagItem → Item ilişkisi ise restrictive deletion kullanır.

Bu sayede bir Item halen bir Bag içerisinde kullanılıyorsa doğrudan silinemez.

Böylece database application kodundan bağımsız olarak domain integrity'yi korur.

24. Önemli Database Constraint'leri

Önemli business invariant'lar database seviyesinde tanımlanmıştır.

Unique User Email

User.email UNIQUE

Bag İçerisinde Unique Item

BagItem(bagId, itemId) UNIQUE

Unique Idempotency Operation

IdempotencyKey(userId, operation, key) UNIQUE

Bu constraint'ler birden fazla application instance aynı anda çalışırken dahi invalid state oluşmasını önler.

25. Neden PostgreSQL?

PostgreSQL bu domain için güçlü bir seçimdir çünkü domain ilişkisel yapıdadır.

Uygulama özelliklerle:

- ACID transactions
- unique constraints
- foreign keys
- referential integrity
- transaction isolation
- concurrency control
- JSON support

özelliklerinden faydalanmaktadır.

Özellikle Copy Trip operasyonu PostgreSQL transaction garantilerinden doğrudan faydalanmaktadır.

26. Neden Prisma?

Prisma aşağıdaki avantajları sağlamaktadır:

- type-safe database access
- schema-driven development
- migration support
- transaction support
- explicit relational modeling
- structured database errors

Ayrıca Prisma schema önemli domain constraint'lerinin source code içerisinde açıkça görünmesini sağlar.

27. Neden TypeScript?

TypeScript aşağıdaki katmanlar arasında compile-time type safety sağlar:

- controllers
- services
- schemas
- database access
- API responses

Bu yaklaşım data structure tutarsızlıklarının azaltılmasına ve service layer'ın daha kolay maintain edilmesine yardımcı olur.

28. API Contract'ın First-Class Artifact Olması

OpenAPI specification projenin sonradan eklenmiş bir dokümantasyon parçası değil, API contract'ın temel parçalarından biri olarak ele alınmıştır.

Specification:

- endpoint davranışlarını
- request formatlarını
- response formatlarını
- authentication gereksinimlerini
- idempotency gereksinimlerini
- validation kurallarını
- error response'larını

tanımlar.

Böylece backend, frontend, QA ve API consumer'ları arasında ortak bir contract oluşturulabilir.

29. Tasarım Prensipleri

Implementation aşağıdaki temel prensiplere dayanmaktadır.

Single Responsibility

Controller HTTP sorumluluklarını yönetir.

Service business logic'i yönetir.

Prisma persistence işlemlerini yönetir.

Validation schema'ları request validation'ı yönetir.

Defense in Depth

Önemli kurallar birden fazla katmanda korunmaktadır.

Örneğin:

Application validation

+

Service ownership checks

+

Database constraints

Database as Final Authority

Concurrency-sensitive invariant'lar database tarafından garanti edilir.

Idempotency bunun en önemli örneğidir.

Atomic Operations

Birden fazla resource oluşturan operasyonlar transaction içerisinde çalıştırılır.

Explicit Failure Semantics

Beklenen failure durumları açık application error code'ları ile ifade edilir.

Repeatability

Deterministic seed sayesinde proje aynı başlangıç dataset'i ile tekrar kurulabilir.

30. Production İçin Geliştirme Alanları

Case study kapsamında authentication mekanizması bilinçli olarak basitleştirilmiştir.

Production ortamında aşağıdaki geliştirmeler yapılabilir:

- JWT/OAuth2 authentication
- RBAC/permission management
- rate limiting
- request correlation IDs
- structured logging
- metrics
- distributed tracing
- security headers
- secret management
- database connection pool tuning

- API versioning
- CI/CD
- automated migration deployment
- database backup and recovery
- audit logging
- API gateway integration

Copy Trip için kullanılan temel idempotency tasarımı ise database-level concurrency guarantee üzerine kurulduğu için production mimarisine taşınabilecek sağlam bir temel oluşturmaktadır.

31. Sonuç

Bu implementation yalnızca basit bir CRUD case study olarak ele alınmamıştır.

Özellikle aşağıdaki gerçek dünya problemlerine odaklanılmıştır:

- concurrent requests
- retries
- duplicate operations
- transaction failures
- ownership boundaries
- database consistency

Copy Trip implementation'ı idempotency'nin tek bir teknikten ibaret olmadığını göstermektedir.

Güvenilir bir idempotency çözümü:

Request identity

+

Request hash

+

Database uniqueness

+

Transaction

+

Stored response

+

Explicit conflict handling

+

Automated concurrency tests

bileşenlerinin birlikte çalışmasını gerektirir.

Bu yaklaşım duplicate Trip copy'lerinin oluşmasını engellerken retry ve failure durumlarında da doğru davranışı korur.

Sonuç olarak API katmanı mümkün olduğunca sade tutulmuş, buna karşılık service ve database katmanlarında güçlü consistency garantileri uygulanmıştır.

32. Aday

Okhtay Sattari

Software Developer & Architect

Backend Case Study